Software Requirements Specification

for

Secure File Sharing System

Version 1.0 approved

PREPARED BY

Anjil Tripathi

ROLL NUMBER

202401100200053

INSTITUTION

KIET Group of Institutions, Ghaziabad

DATE

2 September 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Revision History **3**

1. Introduction **4**

1.1 Purpose 4

1.2 Document Conventions 4

1.3 Intended Audience and Reading Suggestions 4

1.4 Product Scope 4

1.5 References 4

2. Overall Description **5**

2.1 Product Perspective 5

2.2 Product Functions 5

2.3 User Classes and Characteristics 5

2.4 Operating Environment 5

2.5 Design and Implementation Constraints 6

2.6 User Documentation 6

2.7 Assumptions and Dependencies 6

3. External Interface Requirements **7**

3.1 User Interfaces 7

3.2 Hardware Interfaces 7

3.3 Software Interfaces 7

3.4 Communications Interfaces 7

4. System Features **8**

4.1 User Authentication and Account Management 8

4.2 Secure File Upload and File Management 9

4.3 Secure File Sharing and Access Control 9

4.4 Secure Download and Audit Logging 9

5. Other Nonfunctional Requirements **10**

5.1 Performance Requirements 10

5.2 Safety Requirements 10

5.3 Security Requirements 10

5.4 Software Quality Attributes 10

5.5 Business Rules 10

6. Other Requirements 10

Appendix A: Glossary 11

Appendix B: Analysis Models 11

Appendix C: To Be Determined List 11

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Secure File Sharing System, Version 1.0. The system is a web-based application that enables authenticated users to upload, store, share, download, and manage files securely. It focuses on protecting files from unauthorized access through authentication, authorization, controlled sharing, secure communication, and activity logging. This SRS covers the complete application at the system level, including user interfaces, server-side services, data storage, security controls, and external interfaces.

1.2 Document Conventions

The document follows the IEEE-style SRS structure provided in the supplied template. The terms “shall” and “must” indicate mandatory requirements, while “should” indicates a recommended behavior. Functional requirements are uniquely identified using the prefix SFS-REQ followed by a number. Security requirements use SFS-SEC, and performance requirements use SFS-PERF. High, Medium, and Low are used for requirement priority.

1.3 Intended Audience and Reading Suggestions

The primary audience is the software engineering project team, including developers, testers, project managers, database designers, and security reviewers. The document is also intended for instructors or evaluators who need to understand the proposed system and verify that its requirements are complete and testable. Readers should begin with Sections 1 and 2 for the system overview, continue with Section 3 for interfaces, Section 4 for detailed functional features, and then review Section 5 for nonfunctional requirements.

1.4 Product Scope

The Secure File Sharing System provides a centralized and controlled mechanism for exchanging digital files between authorized users. Its objectives are to reduce the risk of unauthorized file access, provide controlled sharing permissions, maintain file ownership, and record important file activities. The system is intended for academic, organizational, or small-team environments where users need a simple web interface for secure file exchange without relying on uncontrolled public links. Key benefits include confidentiality, controlled access, traceability, and easier file management.

1.5 References

1. IEEE Software Requirements Specification Template, supplied for this project.
2. OWASP Application Security Verification Standard (ASVS), for general web application security guidance.
3. RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3, for secure network communication concepts.
4. Project-specific database and API documentation to be prepared during implementation.

Note: Specific implementation versions and deployment URLs are subject to the final technology stack selected for the project.

2. Overall Description

2.1 Product Perspective

The Secure File Sharing System is a new, self-contained web application. Users interact with a browser-based client, which communicates with an application server through secure HTTP-based APIs. The application server performs authentication, authorization, file metadata management, sharing-policy enforcement, and audit logging. A database stores user accounts, file metadata, permissions, and activity records, while a file-storage component stores the actual file objects. The major components are therefore: User Interface, Authentication and Authorization Service, File Management Service, Sharing and Access-Control Service, Database, File Storage, and Audit Log.

2.2 Product Functions

The major functions of the system are:

- User registration and secure login/logout.
- Authentication and session management.
- Secure file upload with validation of file name, size, type, and ownership.
- File storage and metadata management.
- Sharing files with selected registered users.
- Granting and revoking permissions such as view/download access.
- Listing files owned by or shared with the current user.
- Secure file download after authorization checks.
- File deletion by the owner subject to system rules.
- Activity/audit logging for important security-sensitive actions.
- Handling invalid requests and unauthorized access attempts safely.

2.3 User Classes and Characteristics

1. **Registered User** — A normal user who can authenticate, upload files, view owned/shared files, download files for which access is granted, and share or revoke access to files they own.
2. **File Owner** — A registered user in the context of a particular file who controls its sharing permissions and can manage or delete the file according to system rules.
3. **Administrator** — An optional privileged role responsible for managing users, monitoring system activity, and handling administrative operations. Administrative capabilities must not automatically expose file contents unless explicitly authorized by the system policy.
4. **Unauthenticated Visitor** — A user who has not logged in. Such a visitor may access only public application pages such as the login or registration interface and shall not access protected files or user data.

2.4 Operating Environment

The system is designed to operate as a web application on modern desktop or mobile browsers. A typical deployment consists of a client device connected to an application server over HTTPS. The server may run on Linux, macOS, or Windows depending on deployment

requirements. A relational or document-oriented database may be used for metadata and account information, and a dedicated local or cloud storage location may be used for file objects. The exact versions of the runtime, database, and storage provider are implementation decisions to be finalized during development.

2.5 Design and Implementation Constraints

The system shall use secure authentication and authorization mechanisms and shall not expose protected file content without an access check. Communication between client and server shall use HTTPS in production. Passwords shall never be stored in plaintext and shall be protected using a suitable password-hashing algorithm. File names and user-provided metadata shall be validated to reduce injection and path-traversal risks. Upload size and type limits shall be configurable. The implementation shall follow consistent API, database, and coding conventions. The design should minimize dependence on a single storage provider where practical.

2.6 User Documentation

The delivered system should include a short user guide covering registration, login, file upload, file sharing, permission management, downloading, and deletion. The project should also provide an administrator/developer guide describing configuration, database setup, environment variables, deployment steps, and basic troubleshooting. Security-sensitive configuration values such as secret keys and database credentials must be documented as environment variables rather than hard-coded.

2.7 Assumptions and Dependencies

Assumptions include: users have access to a reasonably stable internet connection; the server and storage service are available during normal operation; the selected database and storage system provide adequate reliability; users understand that sharing a file grants access according to the selected permission; and the deployment environment can provide HTTPS. The system depends on a web browser, application runtime, database, file-storage mechanism, and secure network communication. If any dependency changes, corresponding implementation or configuration changes may be required.

3. External Interface Requirements

3.1 User Interfaces

The user interface shall provide clear pages for registration, login, dashboard/file listing, upload, file details, sharing/permission management, and account/logout. Protected pages shall not be accessible to unauthenticated users. File lists should show useful metadata such as file name, size, owner, upload date, and available actions. Forms shall display understandable validation and error messages without revealing sensitive internal details. The interface should be responsive and usable on common desktop and mobile screen sizes.

3.2 Hardware Interfaces

The system requires ordinary client and server computing hardware capable of running a modern web browser and the selected server runtime. Server storage capacity shall be sufficient for the configured file quota. No specialized hardware is required for the basic version. If an external storage appliance or cloud storage service is used, the application shall communicate with it through the provider's supported API or storage interface.

3.3 Software Interfaces

The application shall interface with: (1) a database for users, file metadata, sharing permissions, and audit records; (2) a file-storage component for file objects; (3) a server-side runtime and web framework; and (4) an email service only if email-based verification or password recovery is implemented. The application API shall exchange structured request and response data, preferably using JSON for metadata operations. Authentication/session tokens and file identifiers shall be handled securely.

3.4 Communications Interfaces

Client-server communication shall use HTTPS in production. API requests shall follow standard HTTP methods such as GET, POST, PATCH/PUT, and DELETE where appropriate. Authentication credentials shall be transmitted only over encrypted connections. File uploads may use multipart/form-data, while ordinary API requests may use JSON. The server shall return appropriate HTTP status codes for success, validation failure, authentication failure, authorization failure, missing resources, and server errors. Sensitive information shall not be included unnecessarily in responses or logs.

4. System Features

The system features below define the major services required by the Secure File Sharing System. Each feature includes its purpose, expected stimulus/response behavior, and verifiable functional requirements.

4.1 User Authentication and Account Management

4.1.1 Description and Priority

High priority. This feature provides secure registration, login, logout, and authenticated access to protected system resources. It establishes the identity of the user before file operations are permitted.

4.1.2 Stimulus/Response Sequences

1. User opens the registration page → system displays the registration form.
2. User submits valid registration details → system validates the data, creates the account, and confirms successful registration.
3. User submits invalid or duplicate details → system rejects the request and displays a suitable error.
4. User submits valid login credentials → system authenticates the user and creates an authenticated session/token.
5. User submits invalid credentials → system denies authentication without revealing which credential was incorrect.
6. User logs out → system invalidates the active session/token and returns the user to an unauthenticated state.

4.1.3 Functional Requirements

The system shall provide the following functional requirements:

SFS-REQ-01: The system shall allow a new user to register using required account information.

SFS-REQ-02: The system shall reject registration when required fields are missing or invalid.

SFS-REQ-03: The system shall prevent duplicate accounts according to the configured unique identifier.

SFS-REQ-04: The system shall store passwords only in a secure hashed form.

SFS-REQ-05: The system shall authenticate users before granting access to protected file operations.

SFS-REQ-06: The system shall provide logout functionality that invalidates the user's active authentication state.

SFS-REQ-07: The system shall return an appropriate authentication error for invalid login attempts without exposing sensitive details.

SFS-REQ-08: The system shall prevent unauthenticated users from accessing protected file metadata or file content.

SFS-REQ-09: The system shall maintain the identity of the authenticated user for each protected request.

4.2 Secure File Upload and File Management

4.3 Secure File Sharing and Access Control

Stimulus/Response Sequences

1. Owner selects a file and chooses Share → system displays sharing controls.
2. Owner selects a registered recipient and permission → system records the sharing permission.
3. Recipient opens the shared file → system checks whether an active permission exists.
4. Authorized recipient requests download → system permits the operation.
5. Unauthorized user requests the file → system denies access.
6. Owner revokes access → system removes or disables the recipient's permission and subsequent protected requests are denied.

Functional Requirements

1. Only authenticated users may access protected system functions.
2. A user may modify sharing permissions only for files they own or for which an explicitly authorized administrative policy permits management.
3. A recipient may access a shared file only while an active permission exists.
4. Revoking access shall take effect for subsequent authorization checks.
5. Users shall not be able to select arbitrary server filesystem paths for download or deletion.
6. Administrative privileges shall be separated from normal user privileges.
7. Security-sensitive events shall be auditable without recording passwords or unnecessary sensitive content.

Database requirements: the database shall maintain unique user identities, file metadata, ownership information, sharing permissions, and audit records. Referential consistency should be maintained between users, files, permissions, and audit entries. The system should support backup and recovery procedures appropriate to the deployment environment.

Data requirements: each file record should have a unique identifier, original file name, storage reference, size, owner, creation timestamp, and status. Sharing records should identify the file, recipient, permission, creation/revocation state, and relevant timestamps.

Configuration requirements: secrets, database connection strings, storage credentials, encryption keys, and other sensitive configuration values shall be supplied through secure environment/configuration mechanisms rather than source code.

4.4 Secure Download and Audit Logging

Stimulus/Response Sequences

1. User requests a download → system authenticates the user.
2. System checks file existence and permission → if valid, the file is returned securely.
3. If permission is invalid or the file does not exist → system denies the request with an appropriate response.
4. System records significant actions such as upload, share, revoke, download, and delete with timestamp and user identity where applicable.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system should return normal metadata operations within approximately 2 seconds under the expected project load, excluding unusually large file transfers or slow network conditions. File upload and download throughput will depend on network bandwidth and configured storage. The system shall support concurrent authenticated users within the capacity of the selected deployment environment. Performance limits and maximum supported file size shall be configurable.

5.2 Safety Requirements

The system shall prevent accidental data loss as far as reasonably possible through validation, controlled deletion, and appropriate error handling. A failed upload shall not create misleading metadata indicating that a complete file was stored. The application shall fail safely when storage or database operations are unavailable. Users shall receive clear messages for failures without being exposed to internal server details.

5.3 Security Requirements

Security is a primary requirement of the system. The system shall enforce authentication and authorization for protected resources, use secure password hashing, and use HTTPS in production. It shall validate file names and user input, enforce upload limits, protect against unauthorized direct object access, and avoid exposing sensitive implementation details. Session or token handling shall follow secure practices, including expiration and invalidation where supported. Audit logging shall record important security-sensitive operations. The system should also provide rate limiting or account lockout controls for repeated authentication failures if included in the implementation.

5.4 Software Quality Attributes

- **Security:** confidentiality and access control are the highest-priority quality attributes.
- **Reliability:** file metadata and permissions should remain consistent with stored file objects.
- **Usability:** common operations should be understandable to users with basic web experience.
- **Maintainability:** application components should be modular and documented.
- **Testability:** functional and security requirements should be verifiable through automated and manual tests.
- **Portability:** the application should be deployable on common server environments with minimal configuration changes.
- **Scalability:** the architecture should allow storage and server capacity to be increased as usage grows.

5.5 Business Rules

6. Other Requirements

Appendix A: Glossary

Authentication	Process of verifying the identity of a user.
Authorization	Process of determining whether an authenticated user may perform an action.
Audit Log	Record of security-relevant system events.
File Owner	User who owns a particular uploaded file.
File Metadata	Information describing a file, such as name, size, owner, and timestamps.
HTTPS	HTTP communication protected using TLS.
Permission	Rule defining what a recipient is allowed to do with a shared file.
RBAC	Role-Based Access Control.
SFS	Secure File Sharing System.
TLS	Transport Layer Security.

Appendix B: Analysis Models

The following analysis models are recommended for the project and may be added during design documentation:

- Context diagram showing users, application server, database, and file storage.
- Use-case diagram showing Visitor, User, Owner, and Administrator interactions.
- Data-flow diagram covering authentication, upload, sharing, download, and audit logging.
- Entity-relationship diagram containing User, File, Permission/Share, and AuditLog entities.
- Sequence diagrams for login, file upload, file sharing, and secure download.
- State model for file-sharing permission states such as Active and Revoked.

Appendix C: To Be Determined List

TBD-01: Final implementation technology versions for the backend runtime, database, and storage provider.

TBD-02: Final maximum file-size limit and allowed file-type policy.

TBD-03: Whether email verification and password-reset functionality will be included in Version 1.0.

TBD-04: Whether optional encryption-at-rest will be implemented at the application layer or delegated to the storage provider.

TBD-05: Final deployment environment and production monitoring configuration.